

GraphQL Vulnerabilities — Interview Prep Notes

Variants covered: Introspection Exposure | Query Depth/Complexity (DoS) | Batching Attack (Alias/Array) | GraphQL IDOR | Excessive Data Exposure | Resolver Injection (SQLi/NoSQLi/Cmd) | GraphQL CSRF | GraphQL SSRF

1. Description (logical)

- GraphQL = single endpoint (`/graphql`), client defines query shape → schema exposes full data graph → resolvers execute business logic per field.
- Root issue: REST-era controls (rate limiting, auth-per-endpoint, WAF signatures) don't map cleanly to a single flexible endpoint.
- Same taint model as REST (source→sink→sanitizer) but **sink = resolver**, and **query itself becomes attack surface** (depth, complexity, aliasing).

2. OWASP / CWE mapping

Variant	OWASP API Top 10 (2023)	CWE
Introspection Exposure	API9:2023 Improper Inventory Mgmt	CWE-200
Query Depth/Complexity DoS	API4:2023 Unrestricted Resource Consumption	CWE-400, CWE-770
Batching Attack	API4:2023 / API2:2023 Broken Auth (brute force)	CWE-307, CWE-799
GraphQL IDOR	API1:2023 BOLA	CWE-639
Excessive Data Exposure	API3:2023 Broken Object Property Level Auth	CWE-213
Resolver Injection	API8:2023 Security Misconfiguration / Injection (A03:2021)	CWE-89, CWE-943
GraphQL CSRF	A01:2021 Broken Access Control	CWE-352
GraphQL SSRF	A10:2021 SSRF	CWE-918

3. Common variants

- Introspection left enabled in prod
- Query depth/circular query DoS
- Query complexity/cost-based DoS
- Batch query abuse (array-based, alias-based)
- Field-level BOLA/IDOR (missing per-object auth in resolver)
- Excessive data exposure (schema returns more than UI consumes)

- Injection in resolver→DB layer (SQL/NoSQL/OS command)
- CSRF (GET-query GraphQL endpoints, no SameSite/token)
- SSRF via URL-accepting mutations/fields (webhooks, image fetch, federation)
- Mass assignment via input types
- Directive abuse (`@include` / `@skip` logic bypass)

4. Attack surface / entry points

- `/graphql`, `/graphql`, `/graphql-playground`, `/api/graphql`, subscription WS endpoint (`ws://.../graphql`)
- Introspection query (`__schema`, `__type`)
- Query/mutation/subscription root fields
- Nested/recursive relations in schema (self-referencing types)
- Batched array of operations in single POST
- File upload mutations (`multipart/form-data` GraphQL uploads)
- Federated/gateway subgraphs (internal schema stitching)
- Persisted queries / APQ cache poisoning

5. Testing workflow (condensed, per variant)

A. Introspection

- Verify: is `__schema` query resolvable in prod
- Manual: send `{__schema{types{name,fields{name}}}}`
- Burp: repeater + InQL/GraphQL Raider extension → schema dump
- ZAP: GraphQL add-on → import via introspection
- Nuclei: existing templates (`graphql-introspection`, `graphql-playground`) — yes, off-shelf
- Payload: standard introspection query

B. Depth/Complexity DoS

- Verify: server enforces max depth/cost
- Manual: craft deeply nested/circular query (5→50→500 levels)
- Burp: Repeater, incrementally increase nesting, watch response time/CPU
- ZAP: manual scripted request via active scan script
- Nuclei: custom template needed (send nested payload, assert timeout/5xx)
- Payload: recursive self-referencing field query; aliased duplication of expensive field

C. Batching Attack

- Verify: rate-limit/auth bypass via array of operations in one HTTP call
- Manual: send `[{query:login1},{query:login2}...]` for credential stuffing/OTP brute force
- Burp: Intruder on batched JSON array elements (pitchfork), or aliasing (`q1: login(...) q2: login(...)`)
- ZAP: Fuzzer on batch array

- Nuclei: custom template
- Payload: 100s of aliased mutations bypassing per-request rate limit

D. GraphQL IDOR

- Verify: resolver checks object ownership before returning data
- Manual: swap `id` / `userId` arg with another tenant's ID
- Burp: Repeater — same as REST IDOR but through query variables
- ZAP: manual param tamper
- Nuclei: not effective (needs authz context) — custom only, low value
- Payload: `{user(id:"victim_id"){email,ssn}}`

E. Excessive Data Exposure

- Verify: schema/resolver returns fields not rendered in UI (PII, internal flags)
- Manual: query all fields on a type via introspection, request each
- Burp: InQL bulk query generator → diff against UI response
- ZAP: manual
- Nuclei: no
- Payload: `{me{id,email,password_hash,internal_notes}}`

F. Resolver Injection

- Verify: resolver args passed unsanitized to DB/OS call
- Manual: inject `' OR 1=1--`, `{"$gt":""}`, `; whoami` into query variables
- Burp: Repeater/Intruder on variables JSON
- ZAP: active scan on variables field
- Nuclei: generic SQLi/NoSQLi templates (partial fit, needs custom body)
- Payload: classic SQLi/NoSQLi/cmd-injection payload sets, same as REST

G. GraphQL CSRF

- Verify: query executable via GET with query string, no CSRF token/SameSite
- Manual: build HTML form/img tag with GET `?query=...`
- Burp: CSRF PoC generator
- ZAP: manual
- Nuclei: custom
- Payload: state-changing mutation via GET/simple form POST (`Content-Type: text/plain`)

H. GraphQL SSRF

- Verify: field/mutation accepts URL, server fetches it
- Manual: point URL field to `http://169.254.169.254/latest/meta-data/`
- Burp: Collaborator on URL-accepting mutation
- ZAP: OAST add-on
- Nuclei: SSRF templates adaptable
- Payload: cloud metadata URL, internal IP, `file://`, DNS-based OOB

6. Evidence to collect

- Full request/response pairs (query+variables+headers)
- Schema dump (introspection output) if applicable
- Timing/CPU/memory metrics for DoS
- Response diff showing cross-tenant data (IDOR)
- Collaborator/interactsh interaction log (SSRF)
- Screenshot of unauthorized data field in response

7. Good PoC shape

- Minimal reproducible `curl` /Burp Repeater request with query+variables
- Before/after: authenticated as User A, retrieve User B's data by ID swap
- For DoS: response time graph (baseline vs nested query) + server resource spike
- For CSRF: standalone HTML PoC auto-submitting form
- Always end with **business impact statement**, not just technical proof

8. Attack chains

- Introspection → schema mapping → targeted IDOR/injection
- Batching → OTP/credential brute force bypass → account takeover
- Excessive Data Exposure → PII leak → chained with IDOR for mass extraction
- SSRF → cloud metadata → credential theft → lateral movement
- CSRF → state-changing mutation → account takeover/fund transfer
- Depth DoS → resource exhaustion → availability impact org-wide

9. Common false positives

- Introspection enabled but schema has no sensitive fields (informational only)
- Deep nested query rejected cleanly by depth limiter (control working, not a finding)
- "IDOR" that's actually intended public data (e.g., public profile)
- Slow response due to network/test env noise, not real complexity DoS
- SSRF payload timeout due to firewall, not confirmed OOB callback

10. Prerequisites for exploitability

- Introspection: enabled in non-dev environment
- Depth/Complexity: no query cost analysis/depth limiter configured
- Batching: no per-operation rate limiting inside batch array
- IDOR: resolver missing object-level authorization check
- Injection: resolver builds query/command via string concat, no ORM/parameterization
- CSRF: GET query support + no SameSite/CSRF token + state-changing query
- SSRF: server-side fetch of user-controlled URL, no allowlist

11. What attacker achieves

- Full schema/API surface mapping (recon multiplier)
- Mass PII/data exfiltration (excessive exposure + IDOR)
- Account takeover (CSRF/batched brute force)
- Service disruption (DoS via complexity)
- Internal network access/cloud credential theft (SSRF)
- DB compromise (injection)

12. CIA Impact (per variant, with CVSS vector)

Variant	C	I	A	Sample CVSS v3.1 Vector
Introspection Exposure	Low	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N (5.3)
Depth/Complexity DoS	None	None	High	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H (7.5)
Batching Attack	Medium	Medium	Low	AV:N/AC:L/PR:N/UI:N/S:U/C:M... → practically High if ATO
GraphQL IDOR	High	Low	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N (8.1)
Excessive Data Exposure	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N (7.7)
Resolver Injection	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H (10.0)
GraphQL CSRF	Low	High	None	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:H/A:N (8.1)
GraphQL SSRF	High	Medium	Low	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:L (9.1)

13. Typical CVSS severity

- Injection, SSRF (cloud metadata) → **Critical (9.0–10.0)**
- IDOR, Excessive Exposure, CSRF (financial mutation) → **High (7.0–8.9)**
- Batching (context-dependent, ATO chain) → **High**
- Depth/Complexity DoS → **High (availability-only)**
- Introspection alone → **Low/Medium (informational, recon-enabler)**

14. Business impact (financial app context)

- Excessive Data Exposure/IDOR → mass account/PII/transaction leak → **PCI-DSS, GDPR breach notification**, regulatory fines
- Injection → DB compromise → card data/ledger tampering → **PCI-DSS critical failure**
- CSRF/Batching ATO → unauthorized fund transfers → direct financial loss + reputational damage
- DoS → trading/payment platform downtime → SLA breach, revenue loss
- SSRF → cloud credential theft → lateral compromise of core banking infra

- Introspection → accelerates attacker recon → indirect risk multiplier

15. Indicators/signals

- Abnormally large/nested query payloads in WAF/API gateway logs
- Repeated `__schema` / `__type` queries from single IP
- High volume of aliased operations in single POST (batching signature)
- Resolver errors leaking stack traces/SQL syntax
- Sudden CPU/memory spikes correlated with GraphQL endpoint traffic
- Multiple failed auth attempts bundled in one request (batched brute force)

16. Root cause (code/schema)

```
# Excessive exposure / IDOR — no field-level or object-level auth
type User {
  id: ID!
  email: String
  ssn: String          # sensitive field, no @auth directive
}

type Query {
  user(id: ID!): User # resolver fetches by id, no ownership check
}
```

```
// Resolver injection — string concat into query
resolve: (parent, { name }) => db.query(`SELECT * FROM users WHERE name = '${name}'`)
```

- Common thread: **resolver trusts client-supplied ID/args as authorization context**; schema designed for capability, not least-privilege.
- No central authz middleware — checks scattered/missing per resolver.
- No query cost/depth analysis at execution layer.

17. Remediation / secure design

- Disable introspection & GraphiQL in prod (env-gated)
- Enforce **query depth limit + cost/complexity analysis** (e.g., `graphql-depth-limit`, `graphql-cost-analysis`)
- Disable/limit batching; per-operation rate limiting inside batch
- Object-level authz in **every resolver** (or centralized via directive: `@auth`, `@hasRole`)
- Field-level authz for sensitive fields (`@auth(requires: OWNER)`)
- Parameterized queries/ORM in resolvers — never string concat

- CSRF: disable GET queries for mutations, enforce `Content-Type: application/json` + CSRF token/SameSite=Strict
- SSRF: allowlist outbound URLs, block metadata IP ranges, disable redirects
- Persisted queries / APQ with allowlisting in prod (blocks arbitrary query construction)

18. Compensating controls (if root fix delayed)

- WAF/API gateway rules: block `__schema`, cap request body size, depth heuristics
- Rate limit at gateway (per-IP, per-token) regardless of batching
- Egress firewall rules (block metadata IP, internal ranges) — mitigates SSRF
- Log/alert on high-cost queries, batched auth calls
- Feature-flag/kill-switch for GraphQL endpoint under attack

19. If seen repeatedly → posture signal

- Indicates **no centralized authz layer / schema governance** — same as IDOR pattern (per prior notes) but graph-wide (compounds fast — one missing check exposes entire connected graph, not single endpoint)
- Points to missing **schema review in SDLC**, no complexity/depth limits by default in framework setup
- Suggests GraphQL adopted without adapting AppSec tooling (WAF/rate-limiter still REST-shaped)

20. Upstream improvements

- SAST: Semgrep taint rule — resolver arg → DB/OS sink without sanitizer
- Linter: schema linter enforcing `@auth` directive presence on sensitive types/fields
- Framework: adopt depth-limit + cost-analysis middleware as default in gateway boilerplate
- Design checklist: "every resolver reviewed for object-level authz before merge"
- CI: automated introspection-disabled check for prod config
- Persisted query allowlist enforced at gateway by default

21. Sample triage response

Confirmed: `user(id)` query resolver returns full profile including SSN for arbitrary `id` values without ownership validation — classic BOLA via GraphQL. Verified cross-tenant data retrieval using two test accounts. Severity: High (CVSS 8.1). Recommending immediate authz check in resolver + field-level restriction on `ssn`. Escalating to eng owner for `UserResolver`.

22. Sample developer remediation note

`UserResolver.user(id)` currently trusts client-supplied `id` without verifying requester owns/has access to that object. Add authz check (`ctx.user.id === id` or role-based check) before DB fetch. Additionally, restrict `ssn` field behind `@auth(requires: ADMIN)` directive or separate protected type. Add regression test: authenticated User A must receive 403 when querying User B's `id`. Ref: OWASP API1:2023 BOLA.

23. Real-world CVE / bug bounty examples

- **GitHub GraphQL API** — historical introspection/rate-limit bypass reports via batching in public bug bounty writeups
- **Shopify GraphQL** — publicly disclosed IDOR/BOLA reports via object ID enumeration in GraphQL queries (HackerOne)
- **GitLab** — CVE-2020-13336-class issues; GraphQL endpoint IDOR disclosures (HackerOne reports on gitlab.com scope)
- **Apollo Server / graphql-java** — DoS CVEs tied to unbounded query depth/aliasing before cost-limiting became standard
- General pattern: majority of public GraphQL bug bounty reports (HackerOne/Bugcrowd) fall under introspection info-disclosure + BOLA/IDOR + batching-based brute force